

Task: Matplotlib : All Types of graph

All important components like:

- 1] Title
- 2] labels
- 3] Legend
- 4] Grid
- 5] Color
- 6] Line Style
- 7] Multiline plots
- 8] Marker
- 9] Subplot
- 10] Scatter
- 11] Bar Graph
- 12] Area chart
- 13] Countour pot
- 14] Pie chart
- 15] Histogram

Step 1: import Library

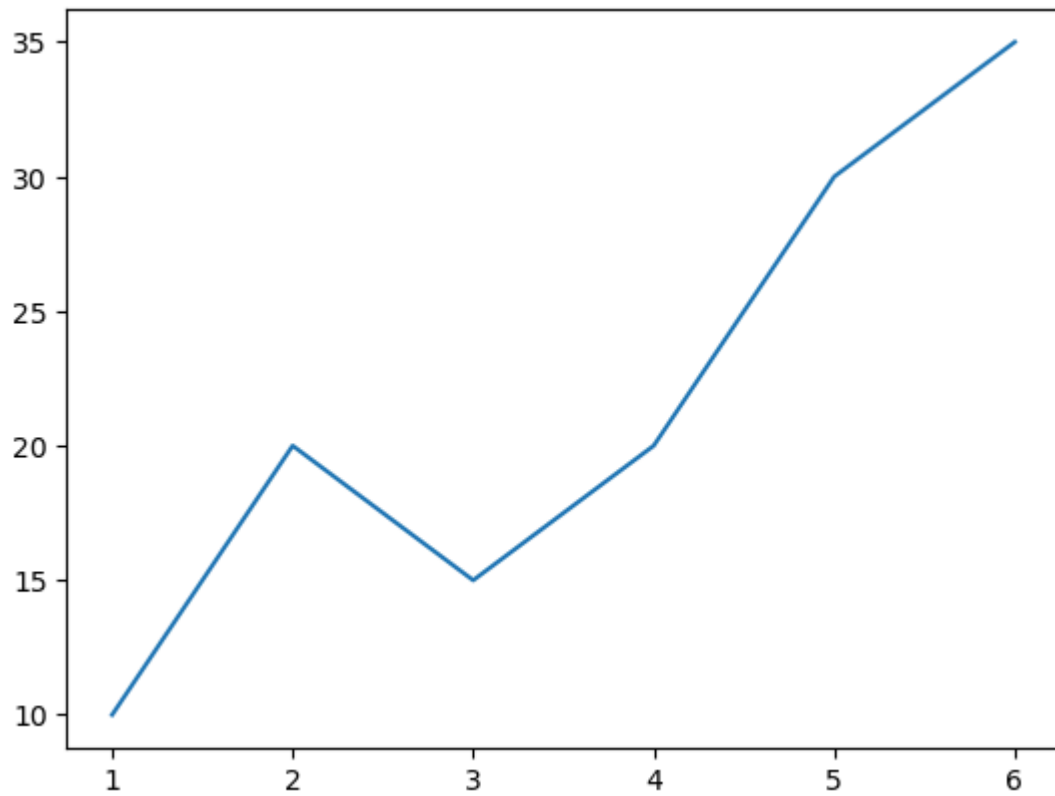
```
In [5]: import matplotlib.pyplot as plt    # used for plotting graphs
```

Step 2: Create data

```
In [6]: x = [1, 2, 3, 4, 5, 6]
        y = [10, 20, 15, 20, 30, 35]
```

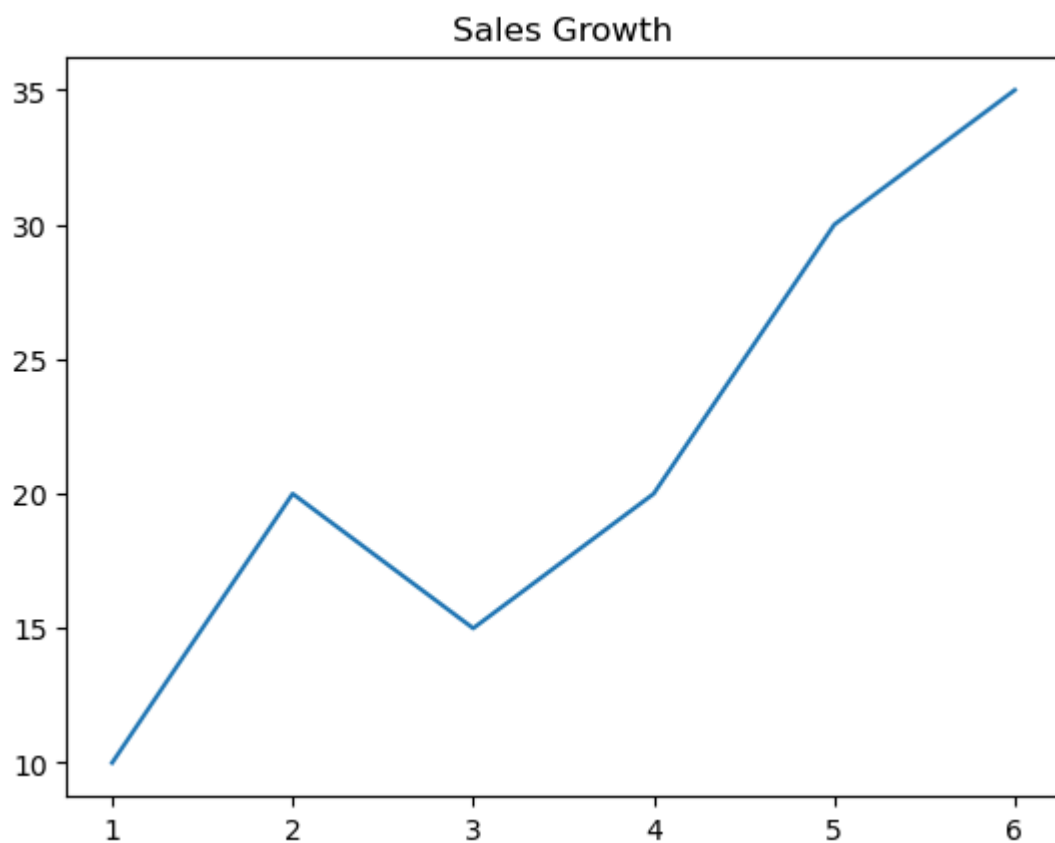
Step 3: Create Basic line graph

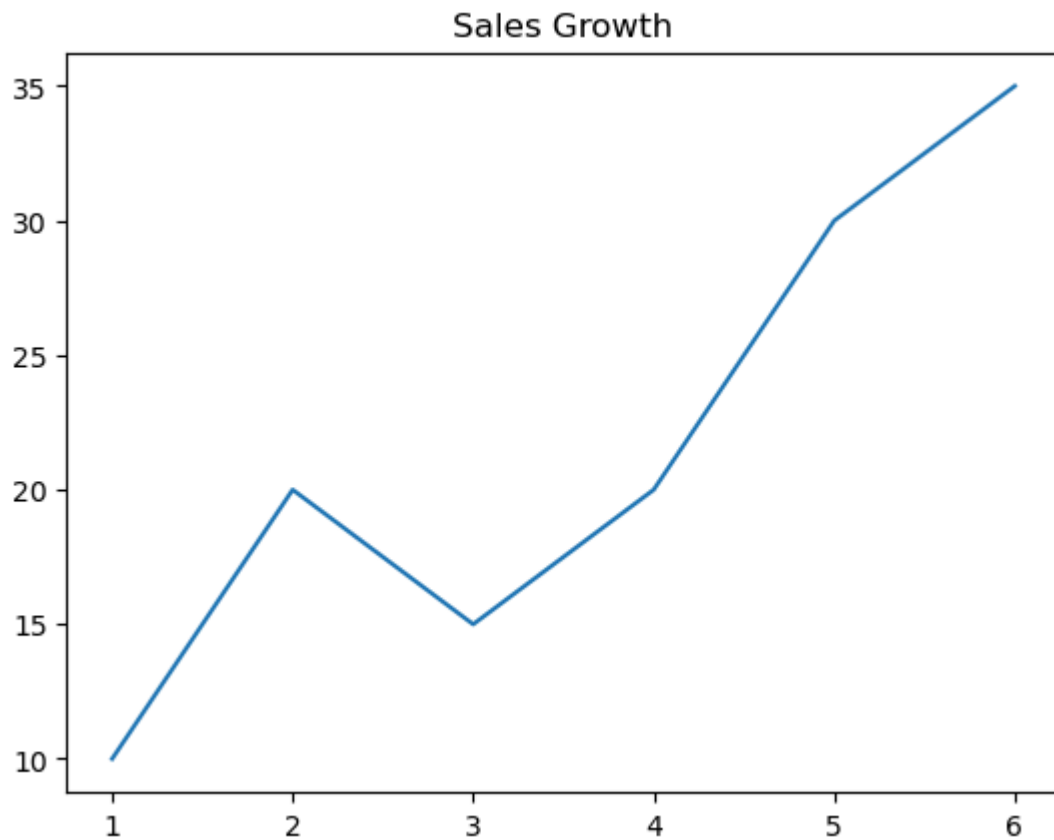
```
In [7]: plt.plot(x,y)    # Simple Line graph.
        plt.show()
```



Step 4: Add Title

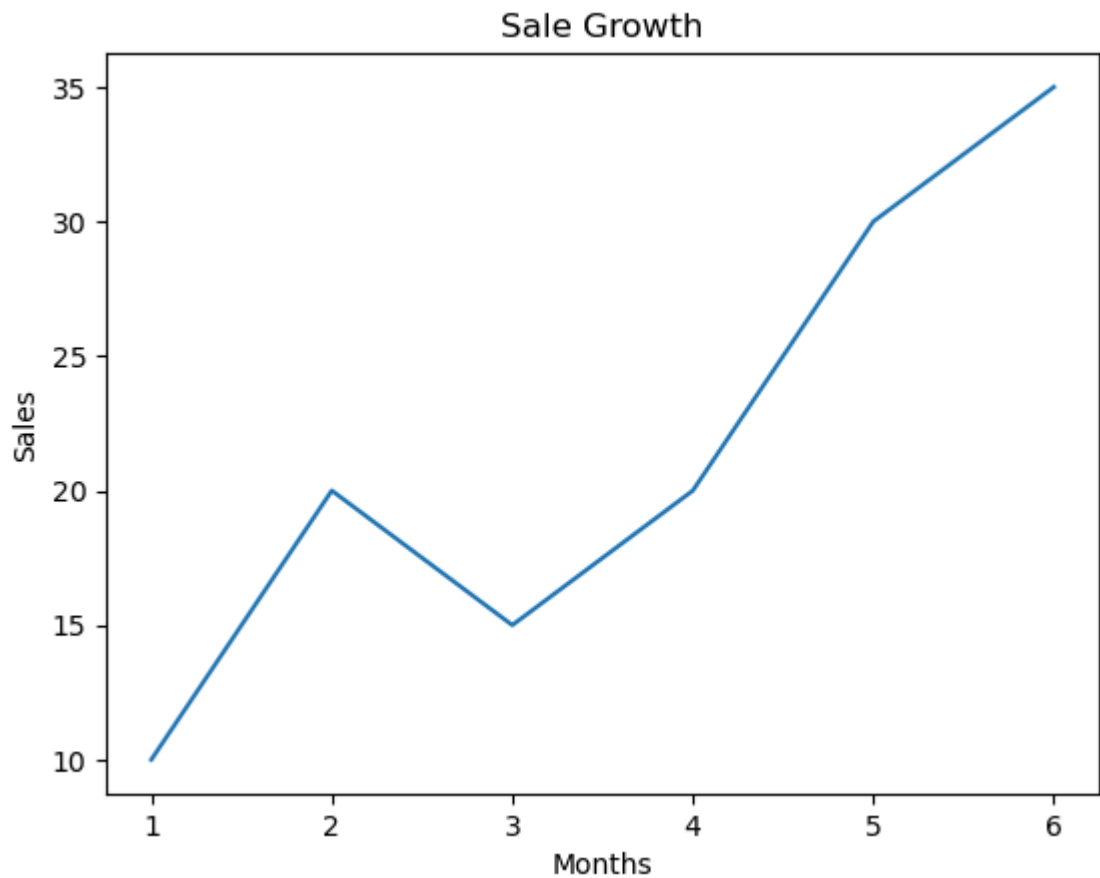
```
In [11]: plt.plot(x,y)                # Title() adds heading to graph.  
plt.title('Sales Growth')  
plt.show()
```





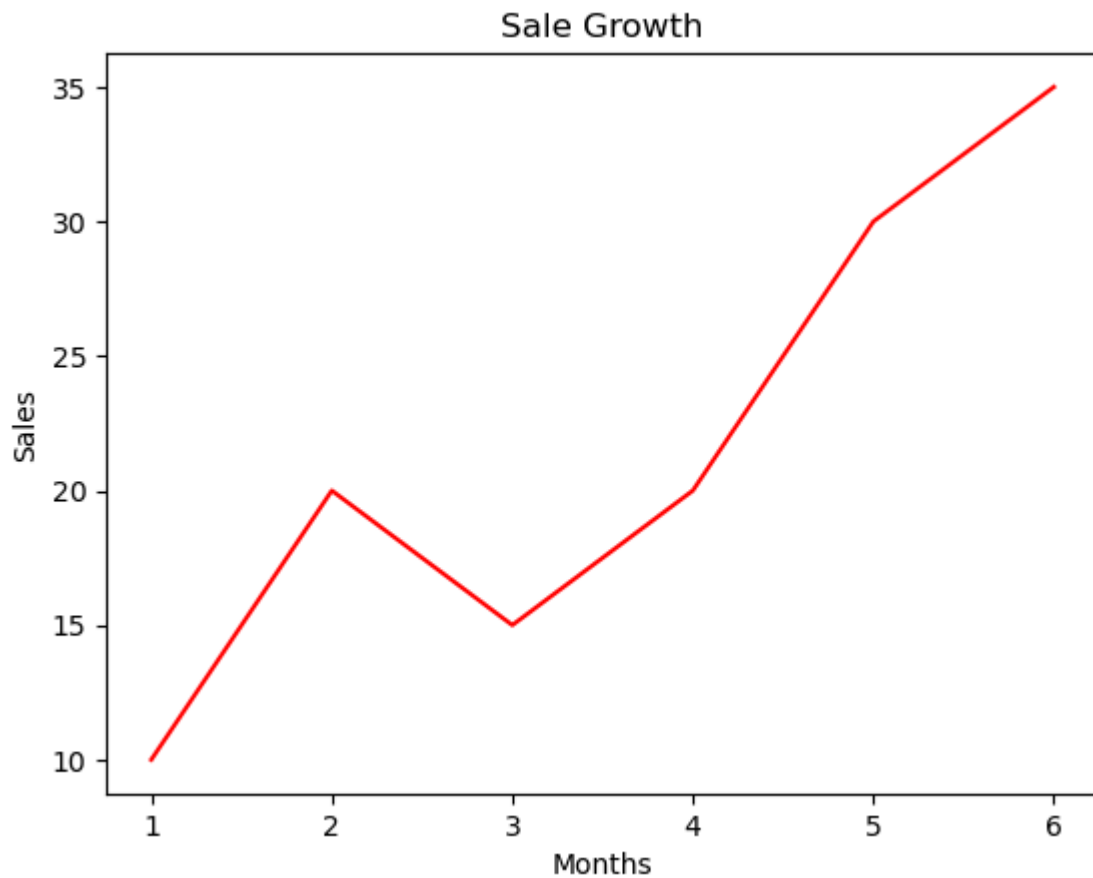
Step 5: Add Labels

```
In [15]: plt.plot(x,y)
plt.title('Sale Growth') # Title of our Graph
plt.xlabel('Months')     # xlabel is name of x axis
plt.ylabel('Sales')      # ylabel is name of y axis
plt.show()
```



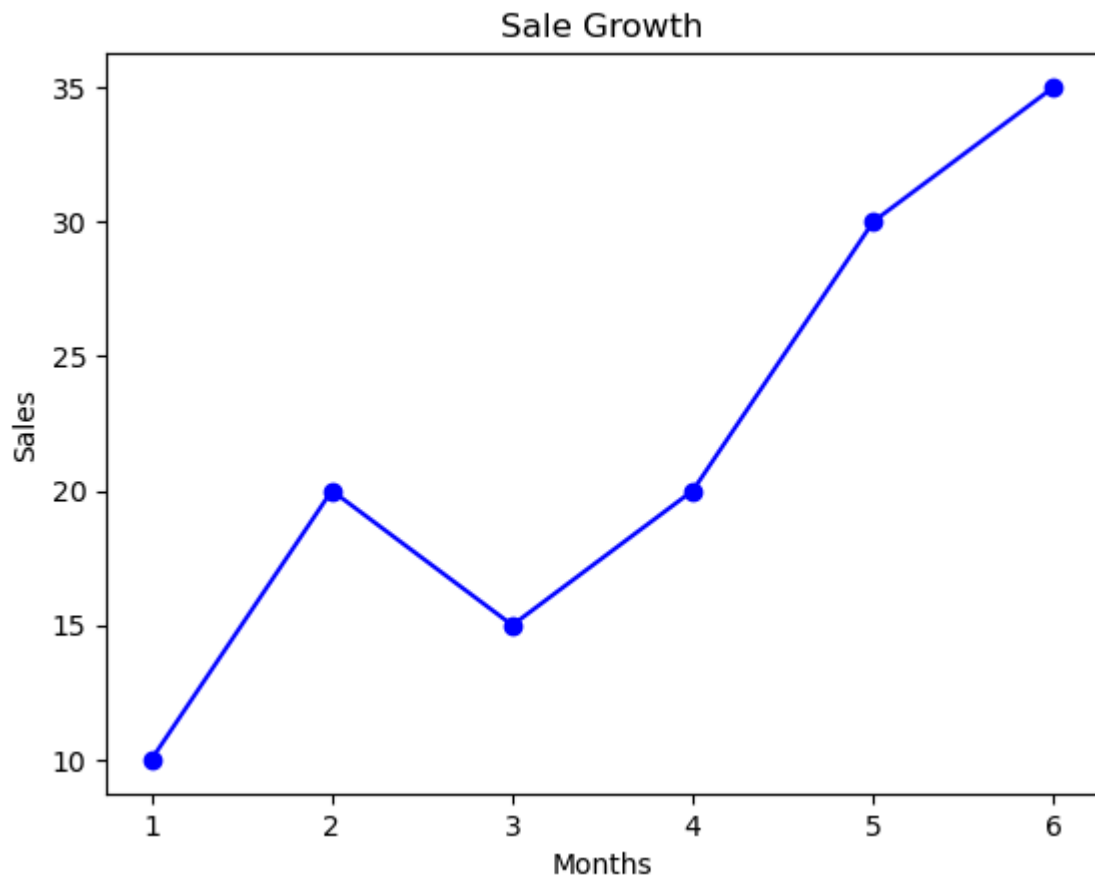
Step 6: Change Line Color

```
In [16]: plt.plot(x,y, color='red')      # Changes line color to 'Red'
plt.title("Sale Growth")
plt.xlabel('Months')
plt.ylabel('Sales')
plt.show()
```



Step 7: Add Marker

```
In [18]: plt.plot(x,y, color='blue',marker = 'o')    # marker 'o' adds circle on line spec
plt.title('Sale Growth')
plt.xlabel('Months')
plt.ylabel('Sales')
plt.show()
```

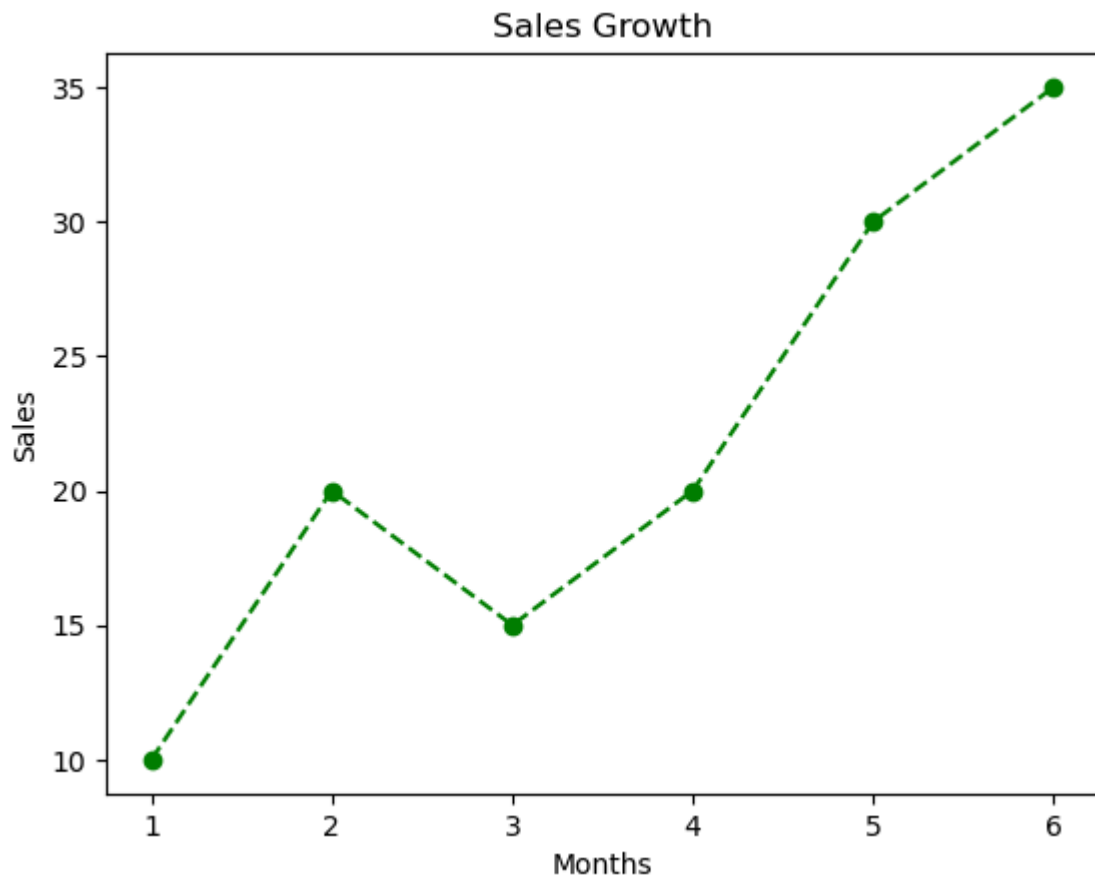


Step 8: Change line Style

```
In [19]: plt.plot(x, y, color='green', linestyle='--', marker='o')

plt.title("Sales Growth")
plt.xlabel("Months")
plt.ylabel("Sales")

plt.show()           # The Line style = '---' create dashed line
```



Multiline plots:

Multiline Plots mean plotting more than one plot on the same figure. We can plot more than one plot on the same figure.

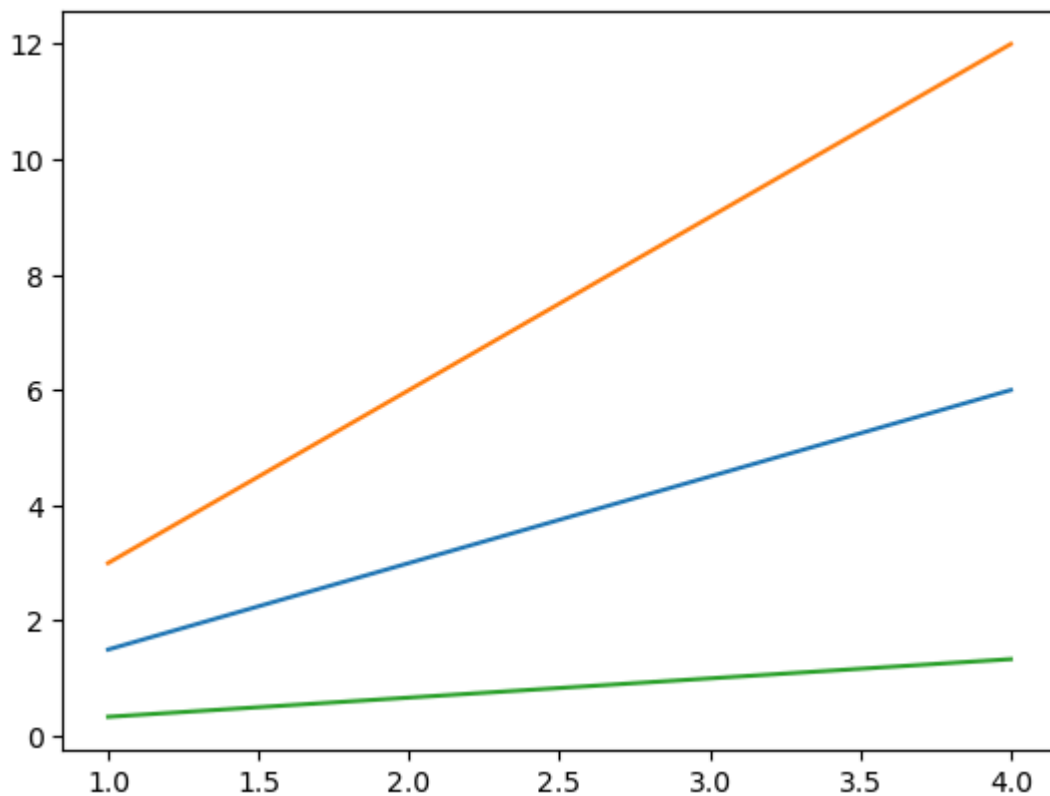
```
In [139... x1= range(1, 5)

plt.plot(x1, [xi*1.5 for xi in x1])

plt.plot(x1, [xi*3 for xi in x1])

plt.plot(x1, [xi/3.0 for xi in x1])

plt.show()
```



Multiline with linear,quadratic,cubic line:

```
In [148... x = np.linspace(0, 2, 100)

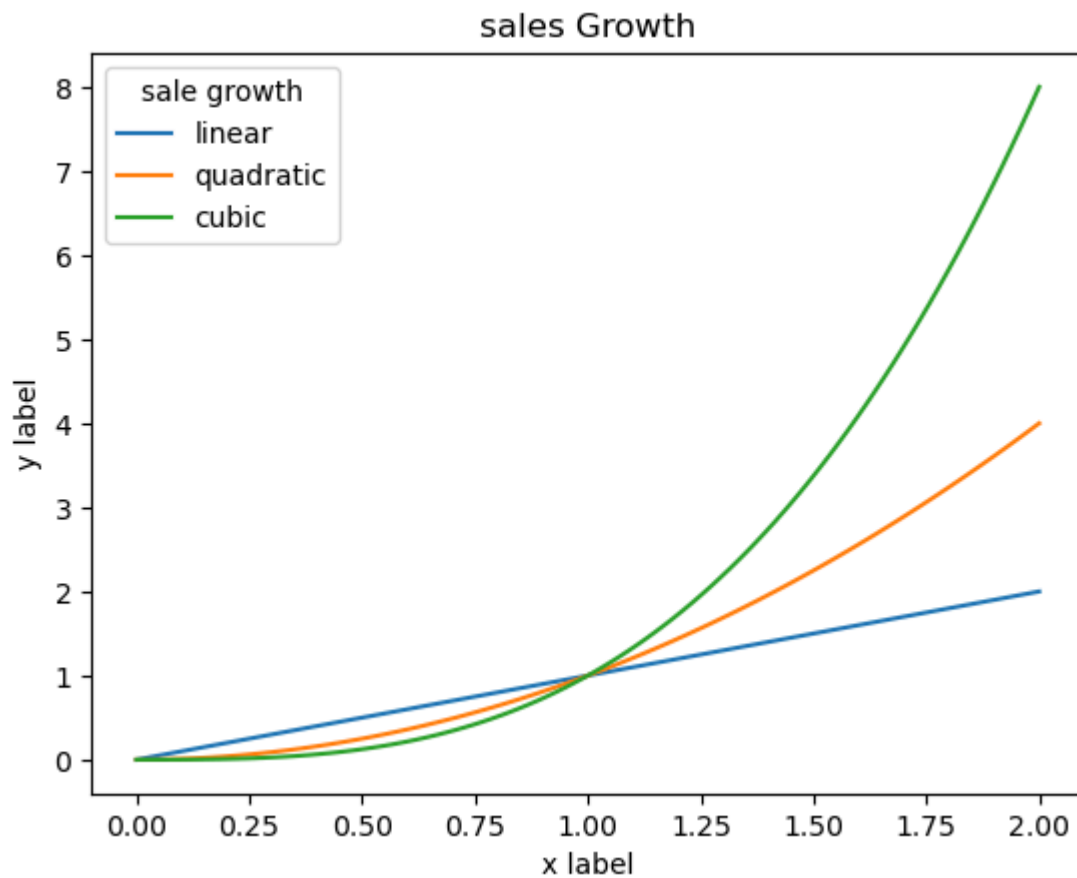
plt.plot(x, x, label='linear')
plt.plot(x, x**2, label='quadratic')
plt.plot(x, x**3, label='cubic')

plt.xlabel('x label')
plt.ylabel('y label')

plt.title("sales Growth")

plt.legend(title = 'sale growth')

plt.show()
```

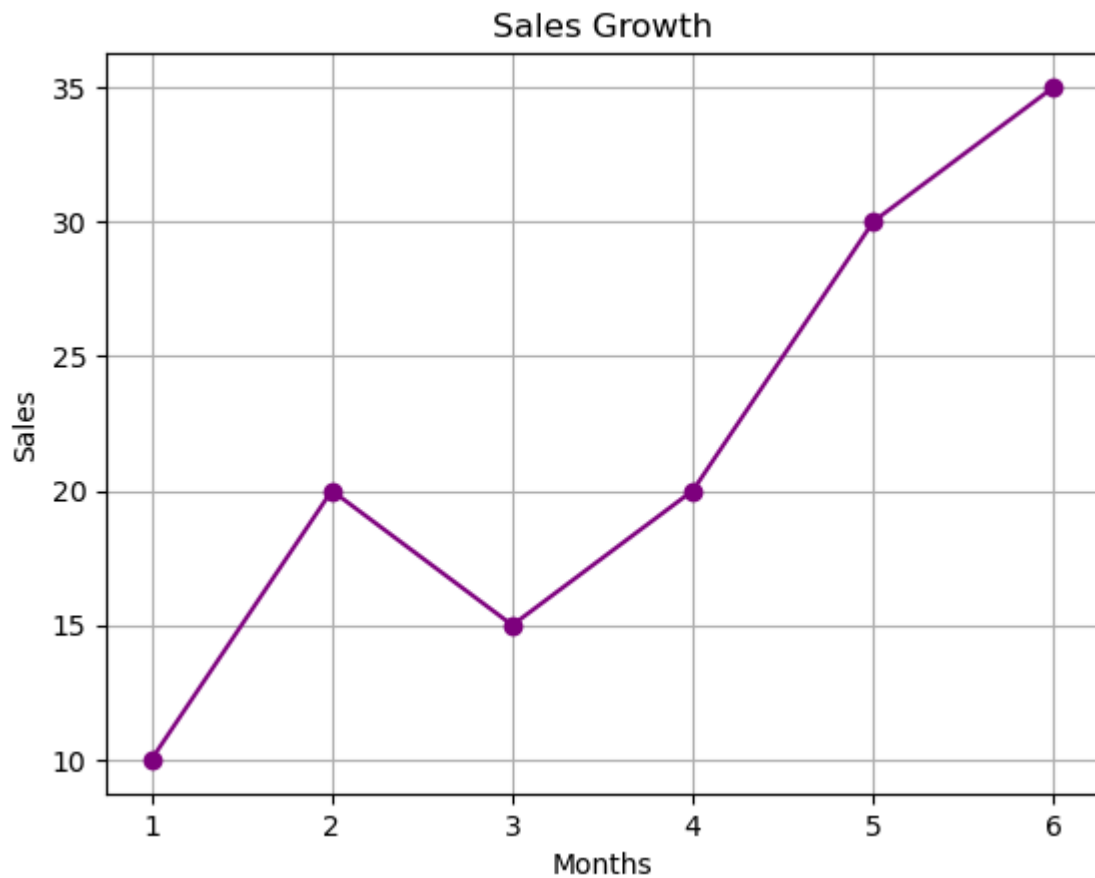
Step 9: Add Grid

```
In [20]: plt.plot(x, y, color='purple', marker='o')

plt.title("Sales Growth")
plt.xlabel("Months")
plt.ylabel("Sales")

plt.grid()

plt.show()           # grid() adds background grid lines.
```



Step 10: Add Legend

```
In [21]: plt.plot(x, y, color='orange', marker='o', label='Sales Data')

plt.title("Sales Growth")
plt.xlabel("Months")
plt.ylabel("Sales")

plt.legend()          # It Gives the name to the data, Displays Label box.

plt.grid()

plt.show()
```



11] Subplot

the subplot() function you can draw multiple plots in one figure:

```
In [27]: import matplotlib.pyplot as plt
import numpy as np

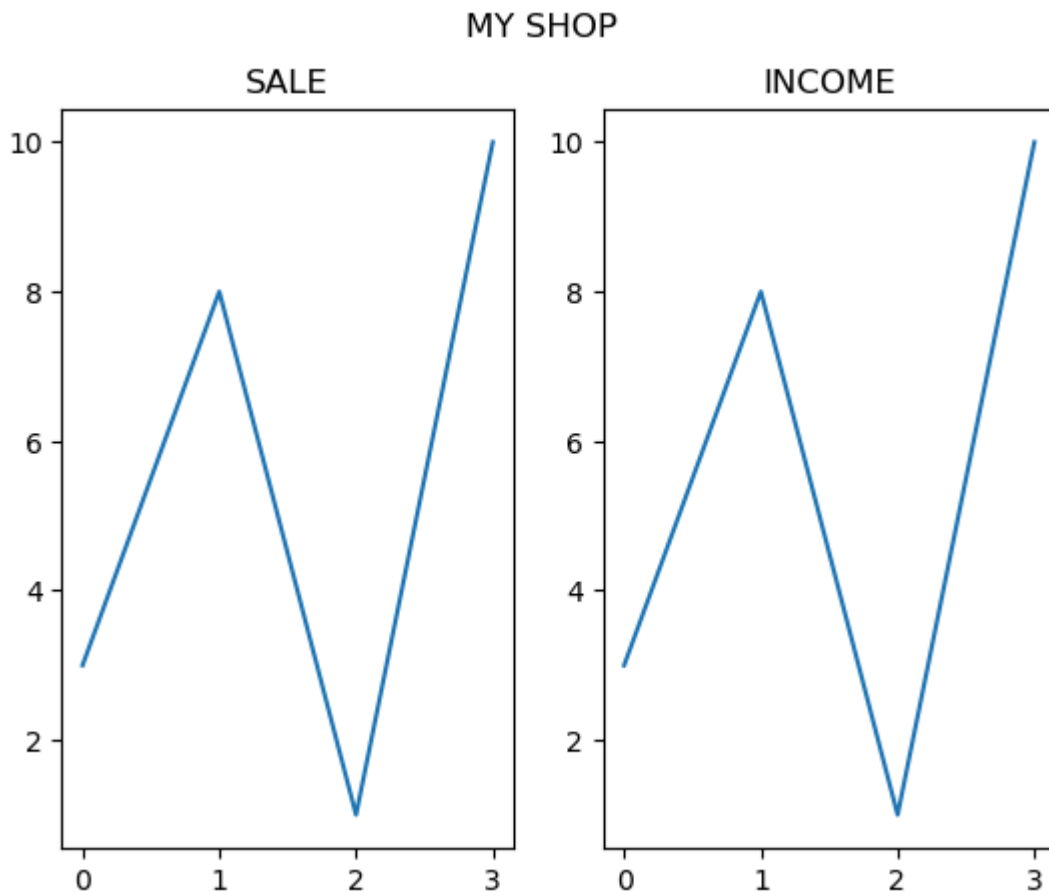
#plot 1:
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

plt.subplot(1, 2, 1)      # the figure has 1 row, 2 columns, and this plot is
plt.plot(x,y)
plt.title('SALE')

# plot 2:
x1 = np.array([0, 1, 2, 3])
y1 = np.array([10, 20, 30, 40])

plt.subplot(1, 2, 2)      # the figure has 1 row, 2 columns, and this plot is t
plt.plot(x,y)
plt.title('INCOME')

plt.suptitle('MY SHOP')
plt.show()
```

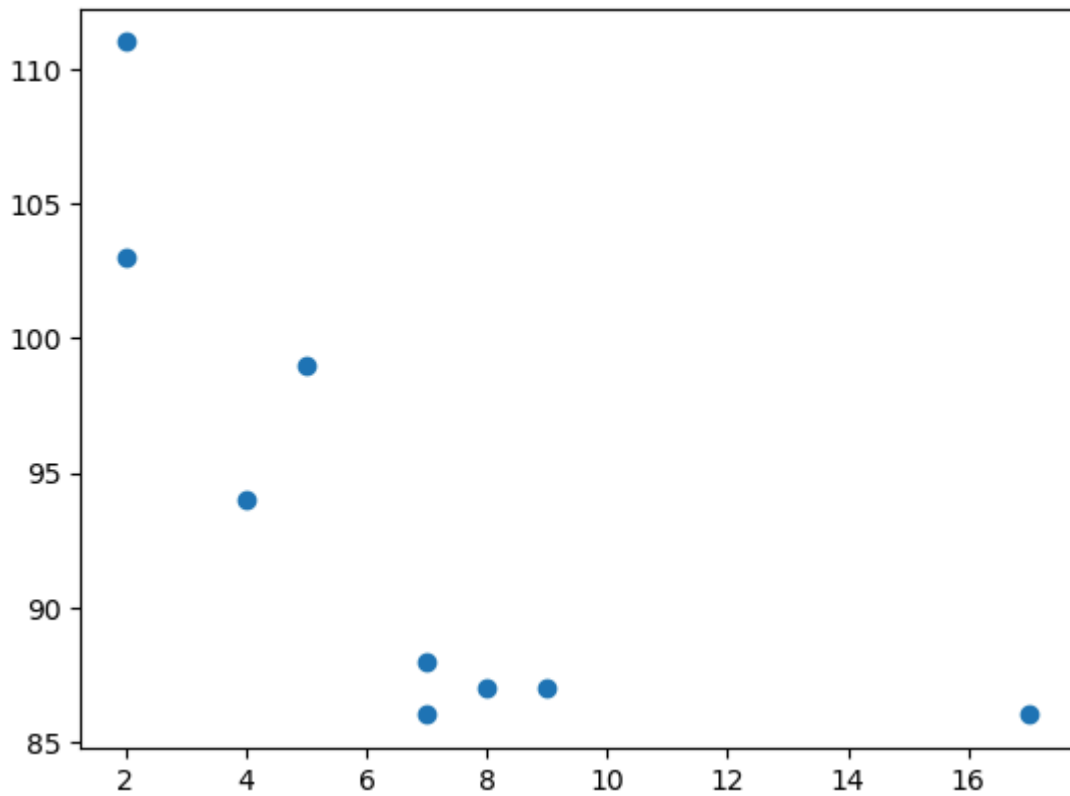


12] Scatter :

The `scatter()` function plots one dot for each observation. It needs two arrays of the same length, one for the values of the x-axis, and one for values on the y-axis:

```
In [34]: x = np.array([5,7,8,7,2,17,2,9,4])
y = np.array([99,86,87,88,111,86,103,87,94])

plt.scatter(x, y)
plt.show()
```

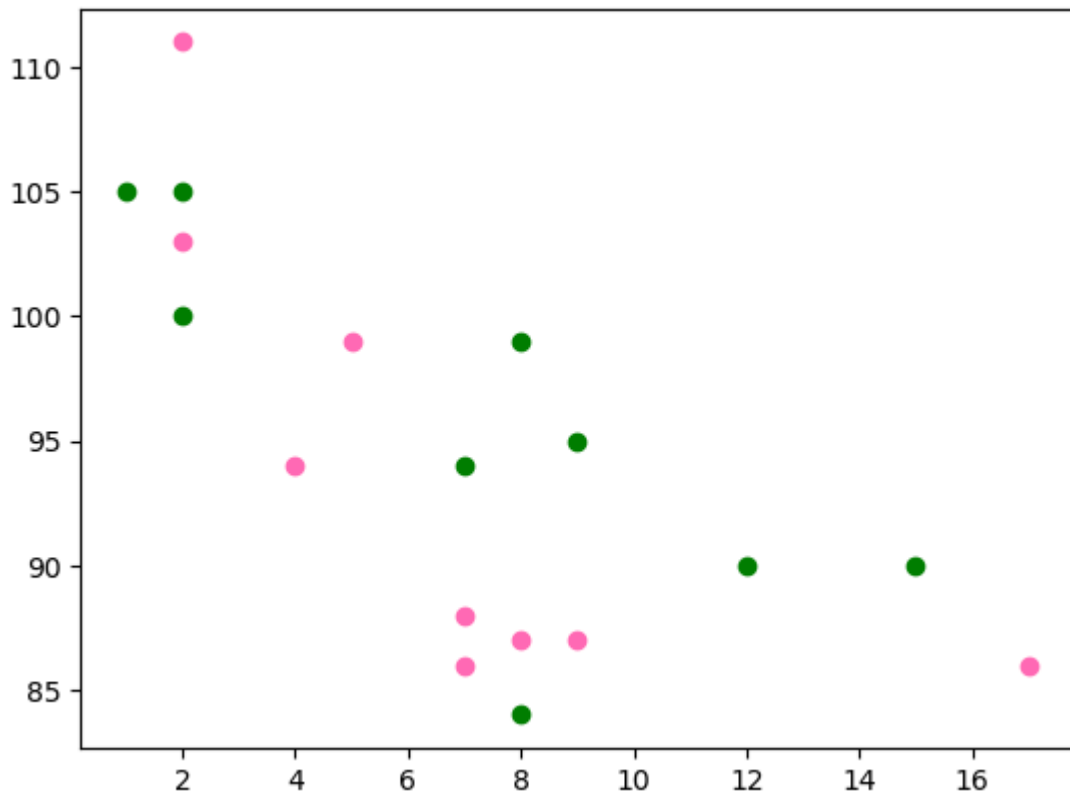


Compare plots with two plot ;

```
In [35]: # day 1, the age and speed of 13 cars:
x = np.array([5,7,8,7,2,17,2,9,4])
y = np.array([99,86,87,88,111,86,103,87,94])
plt.scatter(x, y, color = 'hotpink')

#day two, the age and speed of 15 cars:
x = np.array([2,2,8,1,15,8,12,9,7])
y = np.array([100,105,84,105,90,99,90,95,94])
plt.scatter(x, y, color = 'Green')

plt.show()
```



Color Each Dot

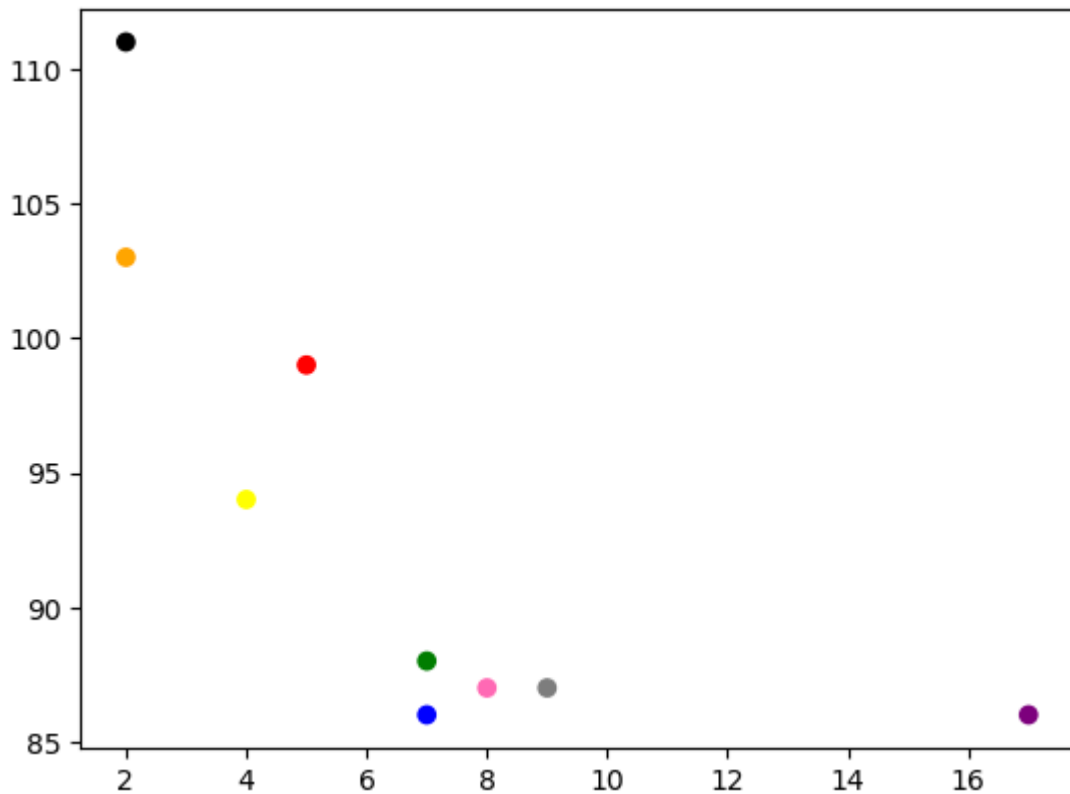
You can even set a specific color for each dot by using an array of colors as value:

```
In [40]: x = np.array([5,7,8,7,2,17,2,9,4])
y = np.array([99,86,87,88,111,86,103,87,94])

color = np.array(['red', 'blue', 'hotpink', 'green', 'black', 'purple', 'orange', 'gray'])

plt.scatter(x, y, c = color)

plt.show()
```



colormap: is called as key " viridis"

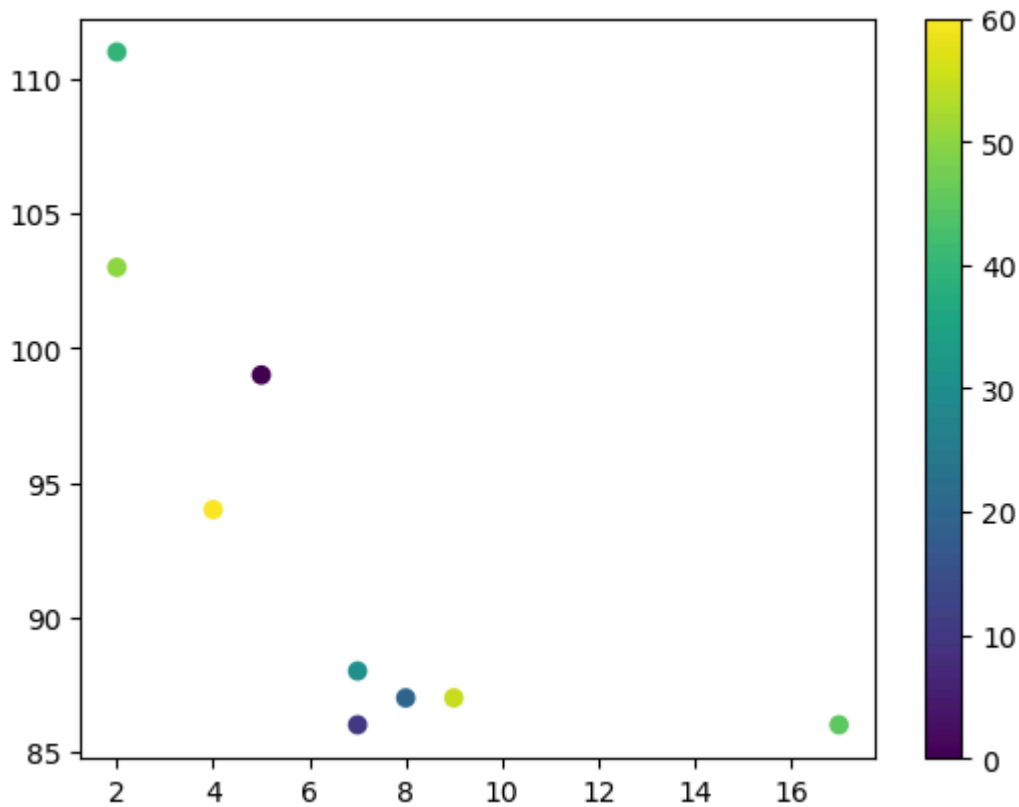
This colormap is called 'viridis' and as you can see it ranges from 0, which is a purple color, up to 100, which is a yellow color.

```
In [41]: x = np.array([5,7,8,7,2,17,2,9,4])
y = np.array([99,86,87,88,111,86,103,87,94])
colors = np.array([0, 10, 20, 30, 40, 45, 50, 55, 60])

plt.scatter(x, y, c=colors, cmap='viridis')

plt.colorbar()

plt.show()
```



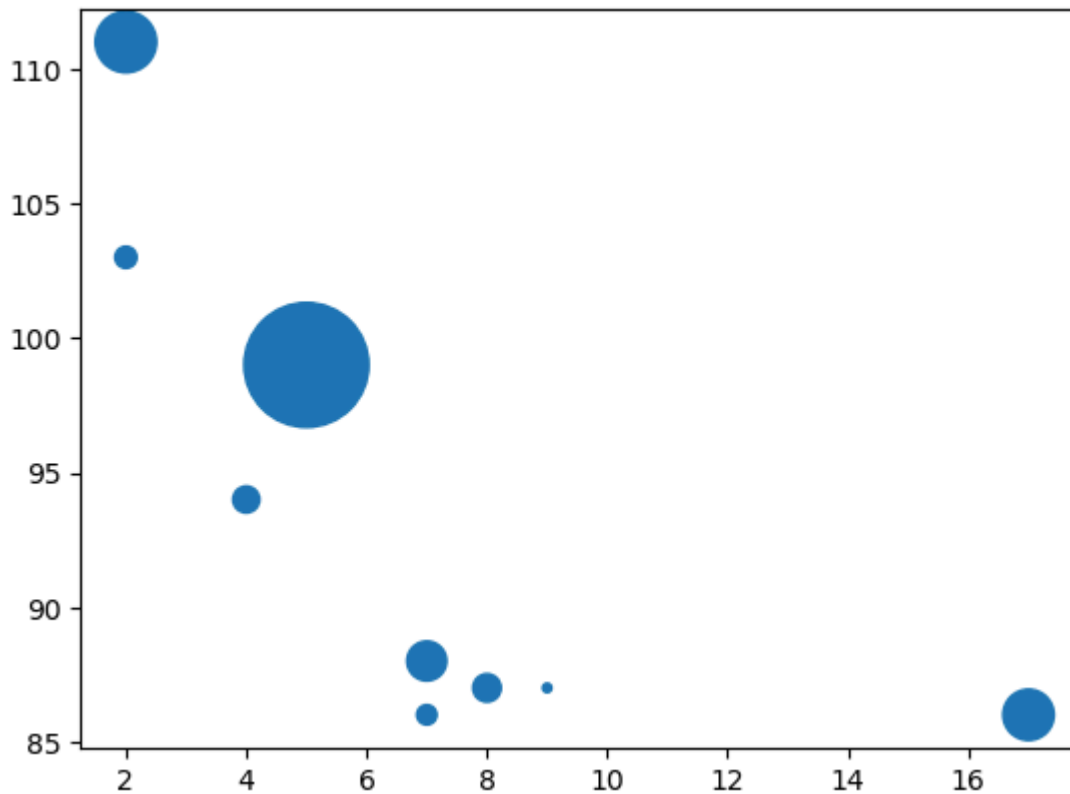
Size :

You can change the size of the dots with the s argument

Note: Just like colors, make sure the array for sizes has the same length as the arrays for the x- and y-axis:

```
In [50]: x = np.array([5,7,8,7,2,17,2,9,4])
y = np.array([99,86,87,88,111,86,103,87,94])
size = np.array([2000,50,100,200,480,333,60,10,90])

plt.scatter(x,y, s = size)
plt.show()
```

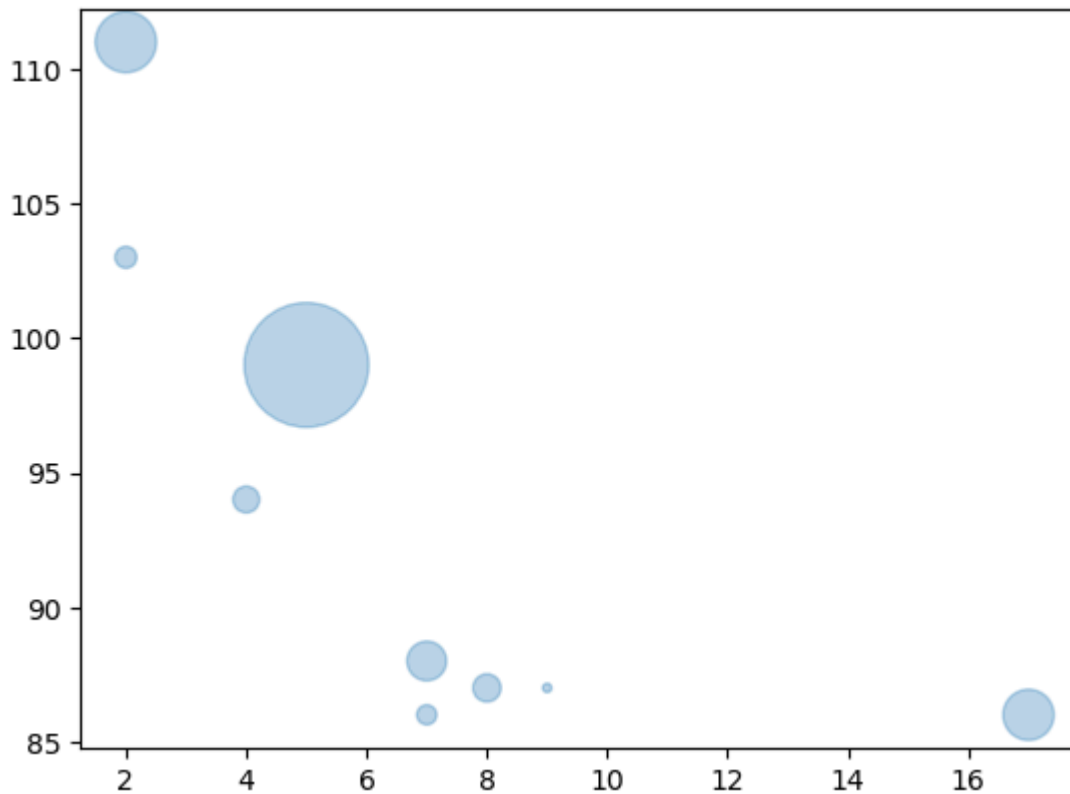
Alpha:

You can adjust the transparency of the dots with the alpha argument

It's Range between 0 to 1 only.

```
In [58]: x = np.array([5,7,8,7,2,17,2,9,4])
y = np.array([99,86,87,88,111,86,103,87,94])
size = np.array([2000,50,100,200,480,333,60,10,90])

plt.scatter(x,y, s = size, alpha = 0.3)
plt.show()
```



Combine Color Size and Alpha

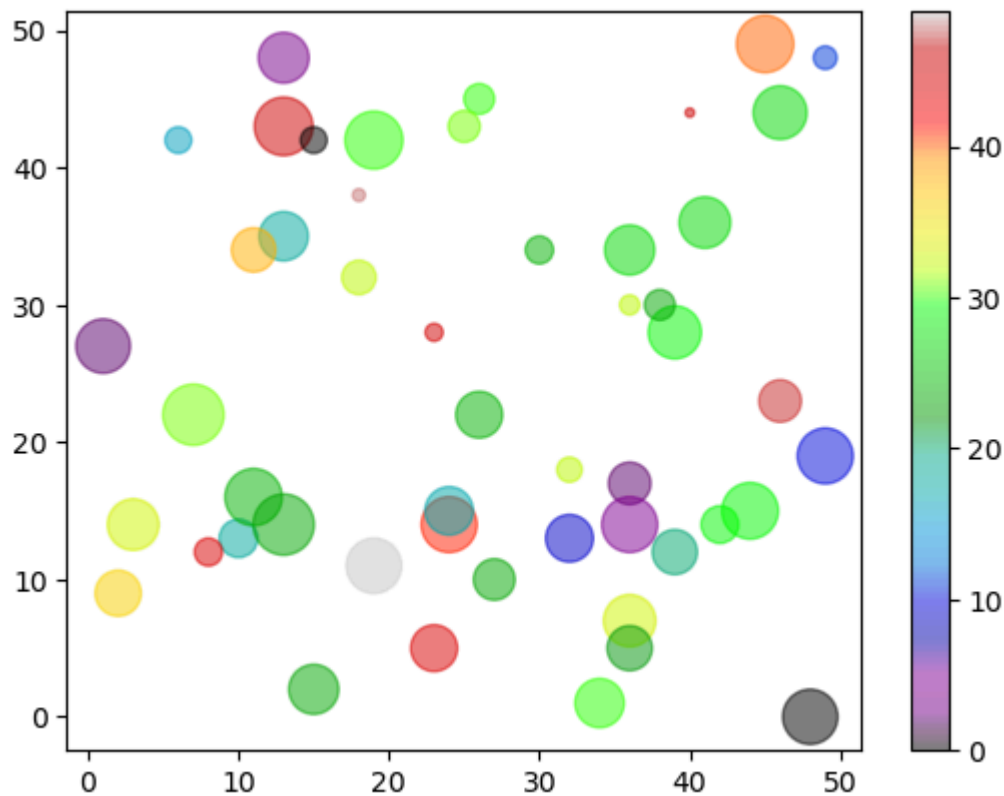
By this combination we get graph with different size, color and with transparent dots

```
In [64]: x = np.random.randint(50, size=(50))
y = np.random.randint(50, size=(50))
colors = np.random.randint(50, size=(50))
sizes = 10 * np.random.randint(50, size=(50))

plt.scatter(x, y, c=colors, s=sizes, alpha=0.5, cmap='nipy_spectral')

plt.colorbar()

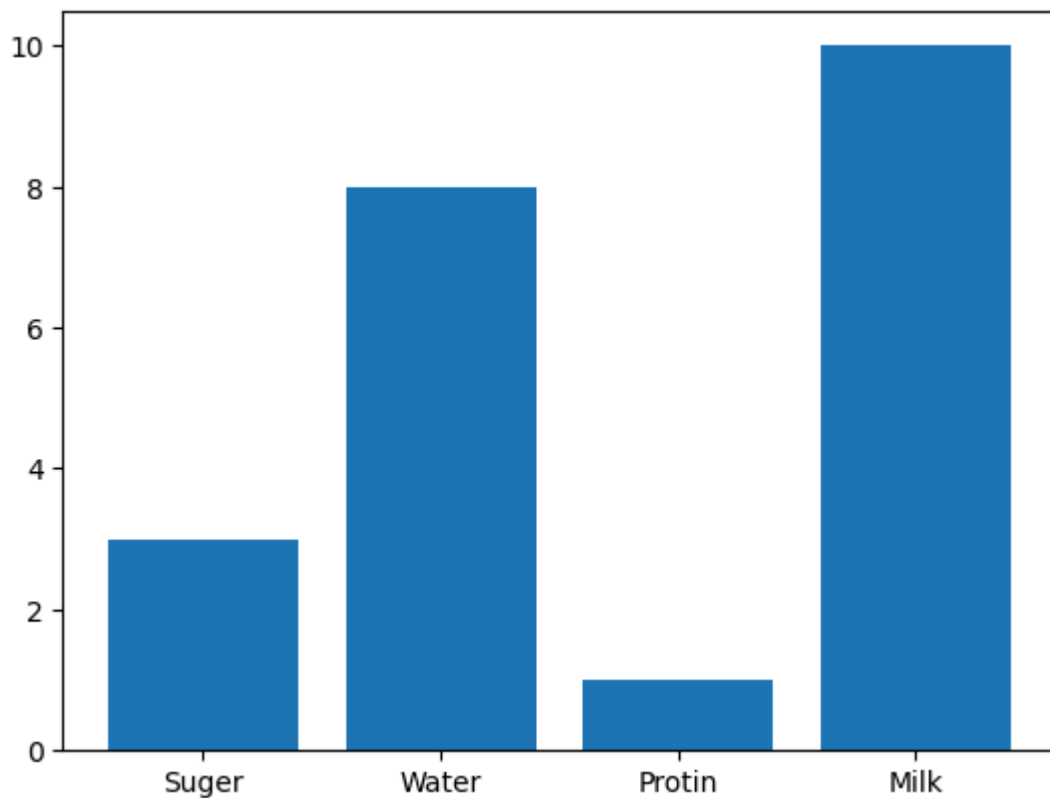
plt.show()
```



Creating Bars :

Vertical Bar

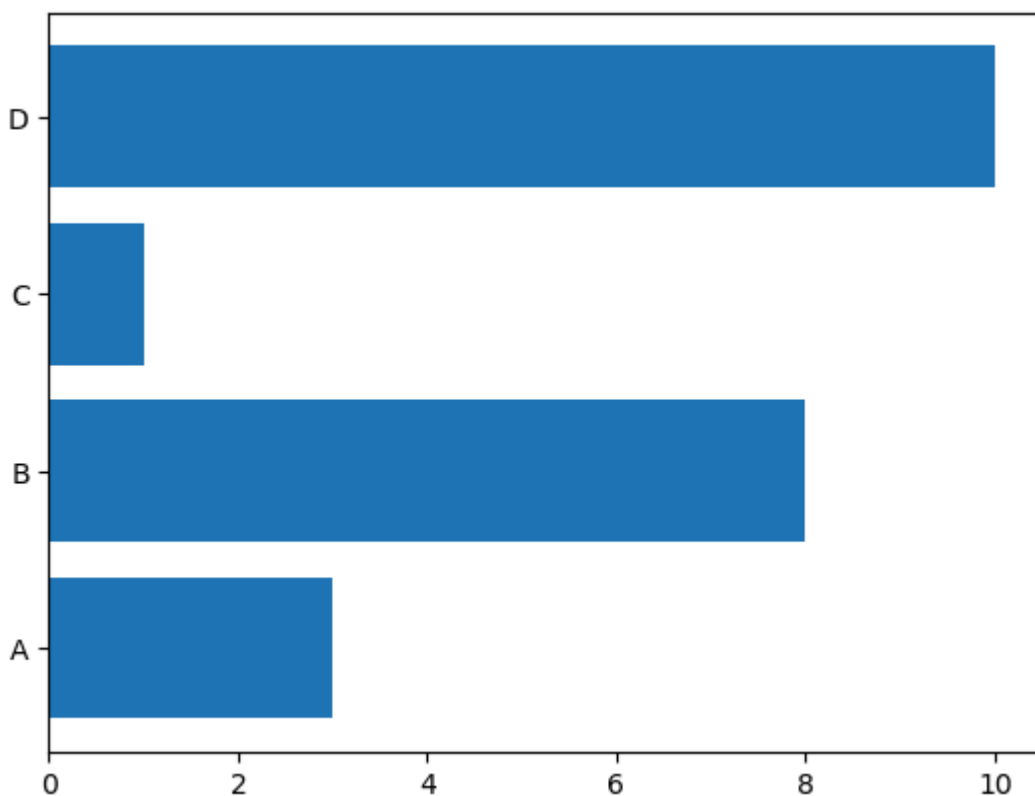
```
In [68]: x = np.array(['Sugar', 'Water', 'Protein', 'Milk'])    # The bar() function takes an array of labels
y = np.array([3, 8, 1, 10])
plt.bar(x, y)
plt.show()
```



Horizontal Bar :

```
In [69]: x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

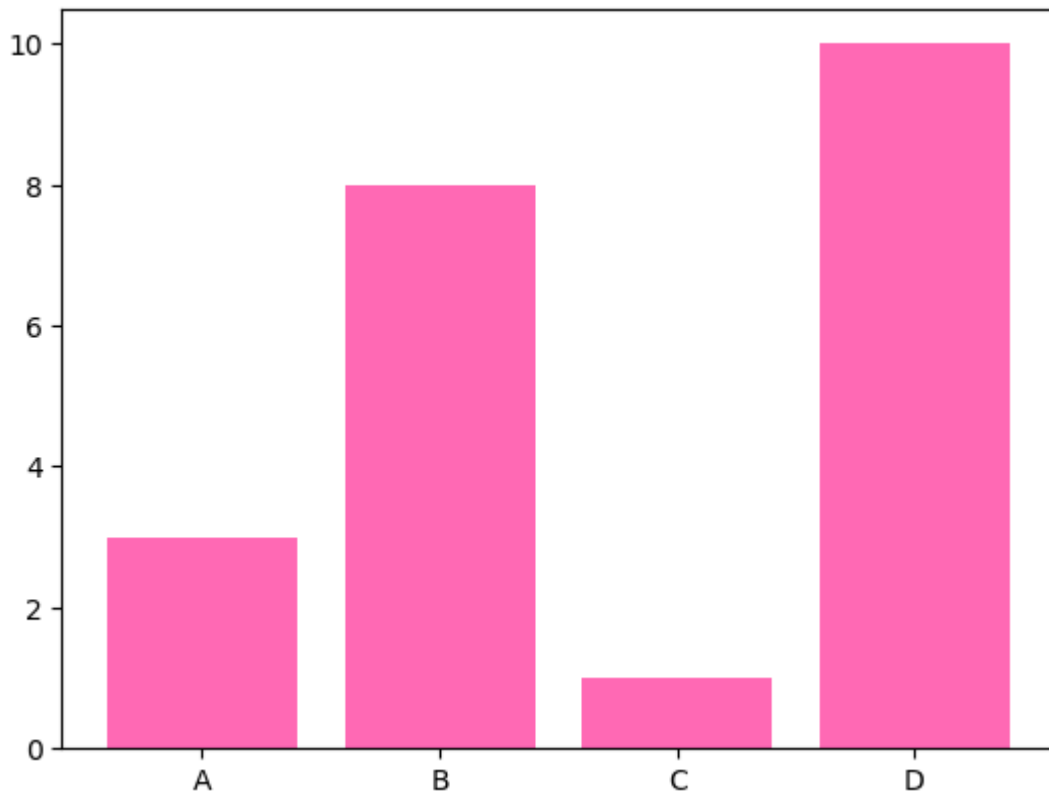
plt.barh(x, y)
plt.show()
```



Bar Color :

```
In [74]: x = np.array(["A", "B", "C", "D"])
y = np.array([3,8,1,10])

plt.bar(x,y, color = "hotpink")
plt.show()
```

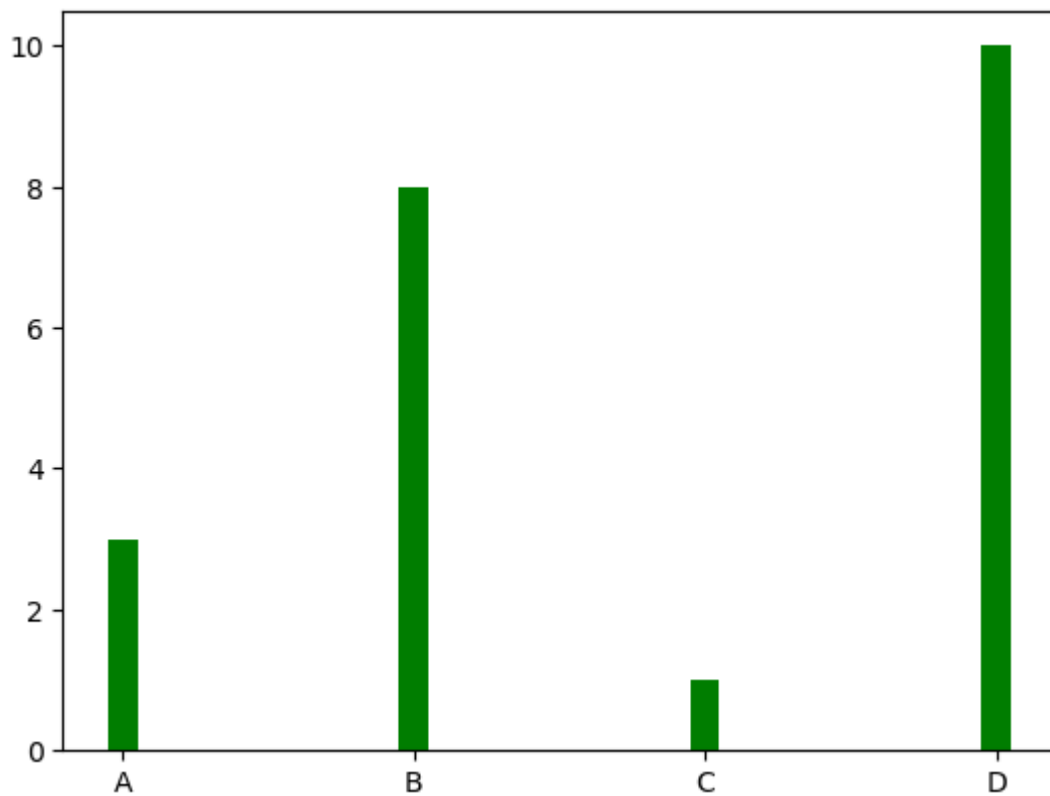


Bar Width :

```
In [79]: x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])

plt.bar(x,y, color = 'green',width = 0.1)

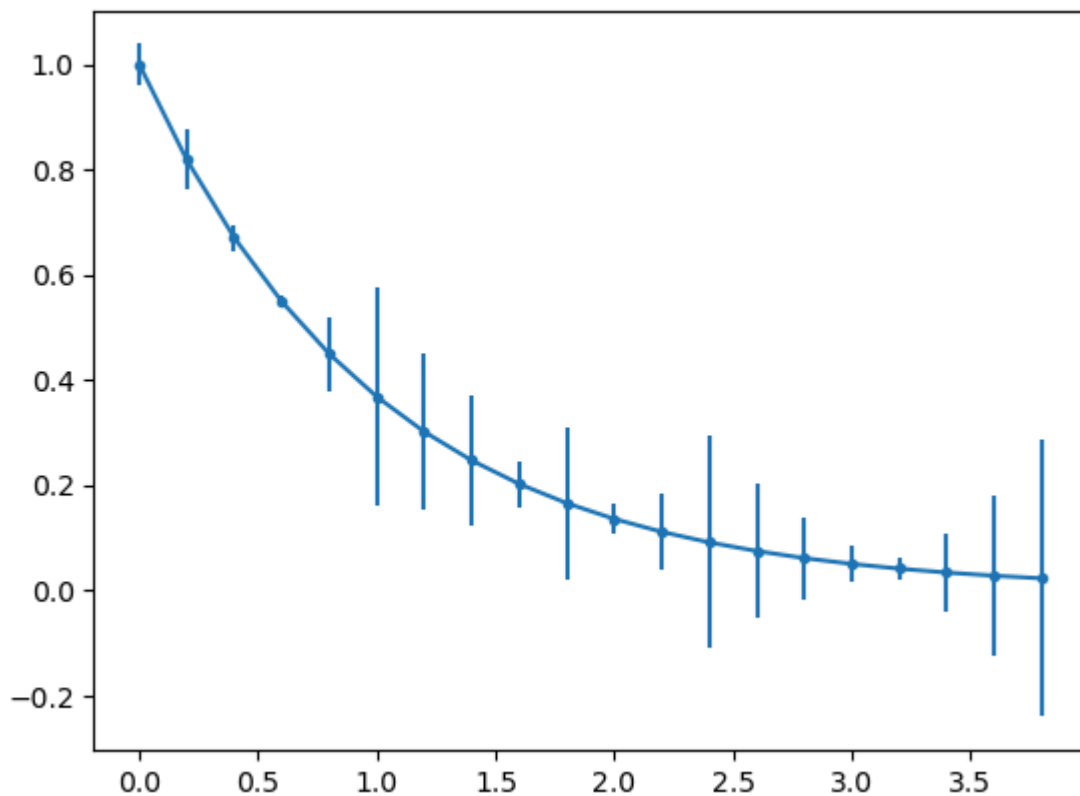
plt.show()
```



Error Bar graph:

In [133...

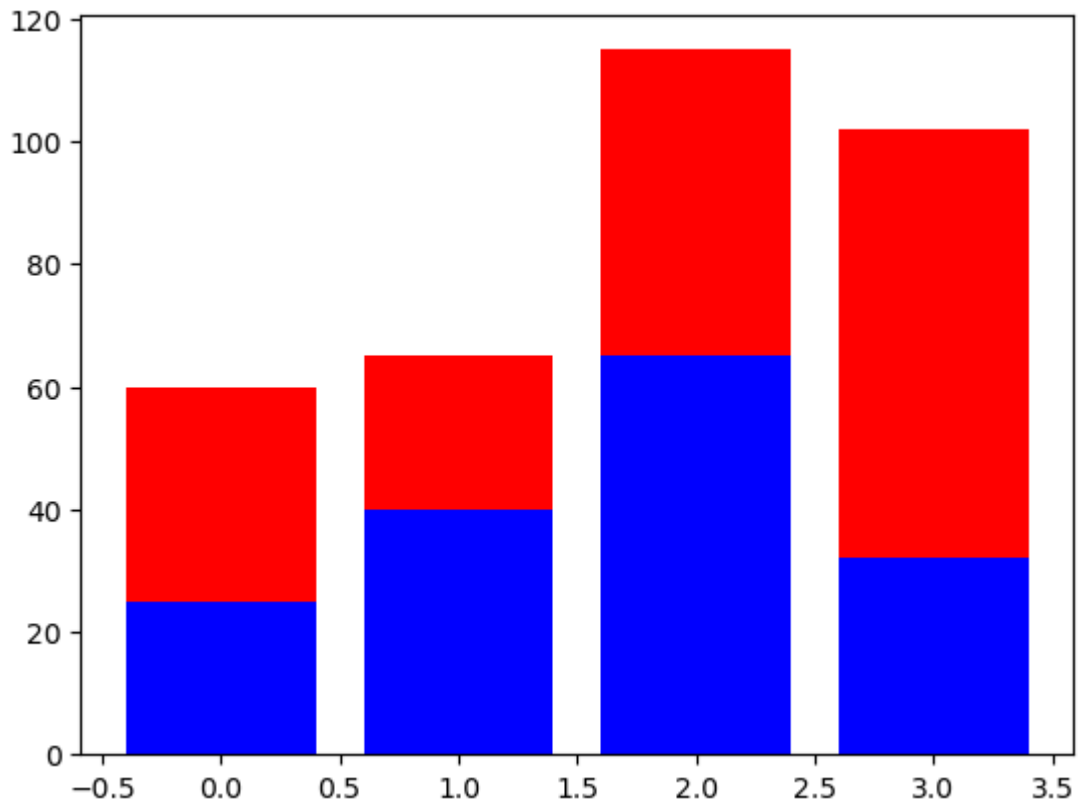
```
x2 = np.arange(0, 4, 0.2)
y2 = np.exp(-x2)
e = 0.1 * np.abs(np.random.randn(len(y2)))
plt.errorbar(x2, y2, yerr = e, fmt = '.-')
plt.show();
```



Stacked Bar graph:

In [134...

```
A = [25., 40., 65., 32.]  
B = [35., 25., 50., 70.]  
z2 = range(4)  
  
plt.bar(z2, A, color = 'b')  
plt.bar(z2, B, color = 'r', bottom = A)  
  
plt.show()
```



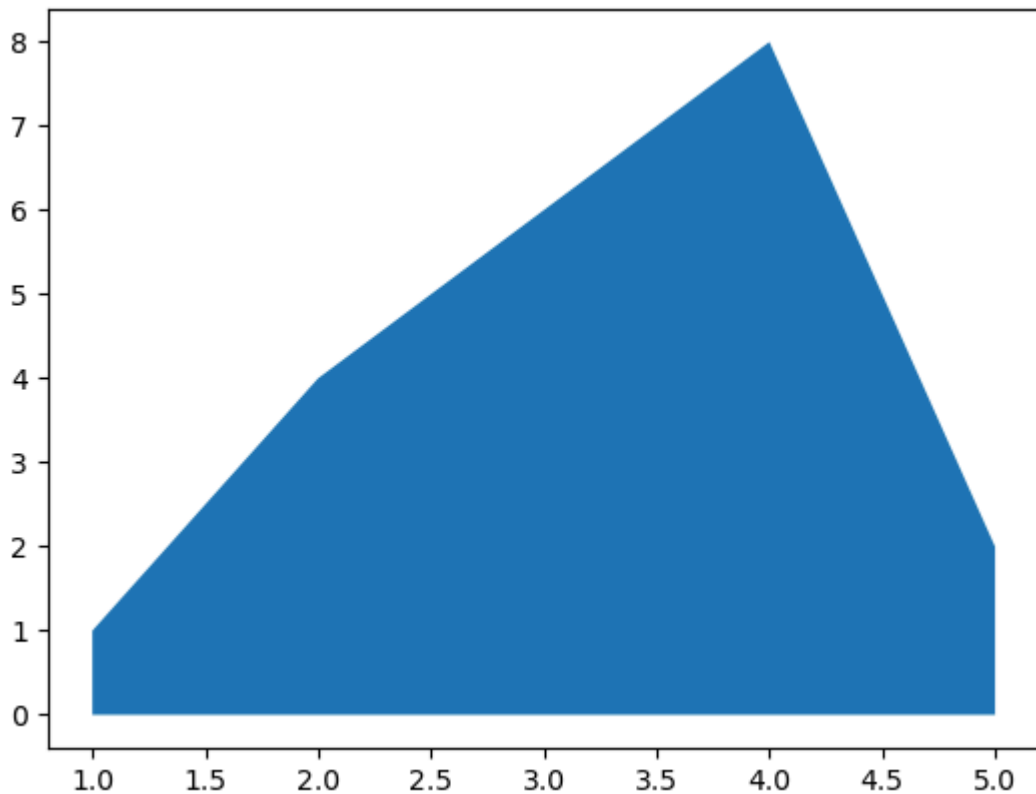
Area chart:

An Area Chart is very similar to a Line Chart. The area between the x-axis and the line is filled in with color or shading.

In [137...

```
x12 = range(1, 6)
y12 = [1, 4, 6, 8, 2]

# Area plot
plt.fill_between(x12, y12)
plt.show()
```

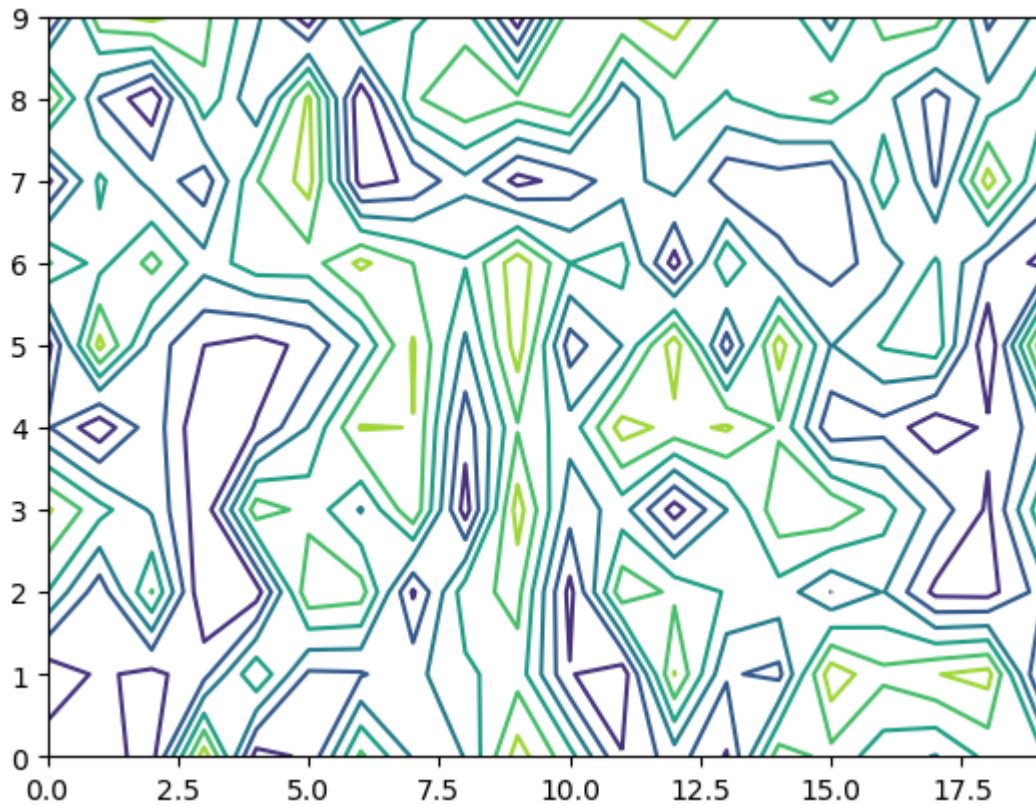
Counter plot:

A contour plot is created using to generate a 2D coordinate grid and evaluated with a function to produce values, which are then visualized using Matplotlib's `plt.contour` or `plt.contourf` to represent 3D surfaces with color-coded elevation lines.

```
In [138... # Create a matrix
matrix1 = np.random.rand(10, 20)

cp = plt.contour(matrix1)

plt.show()
```

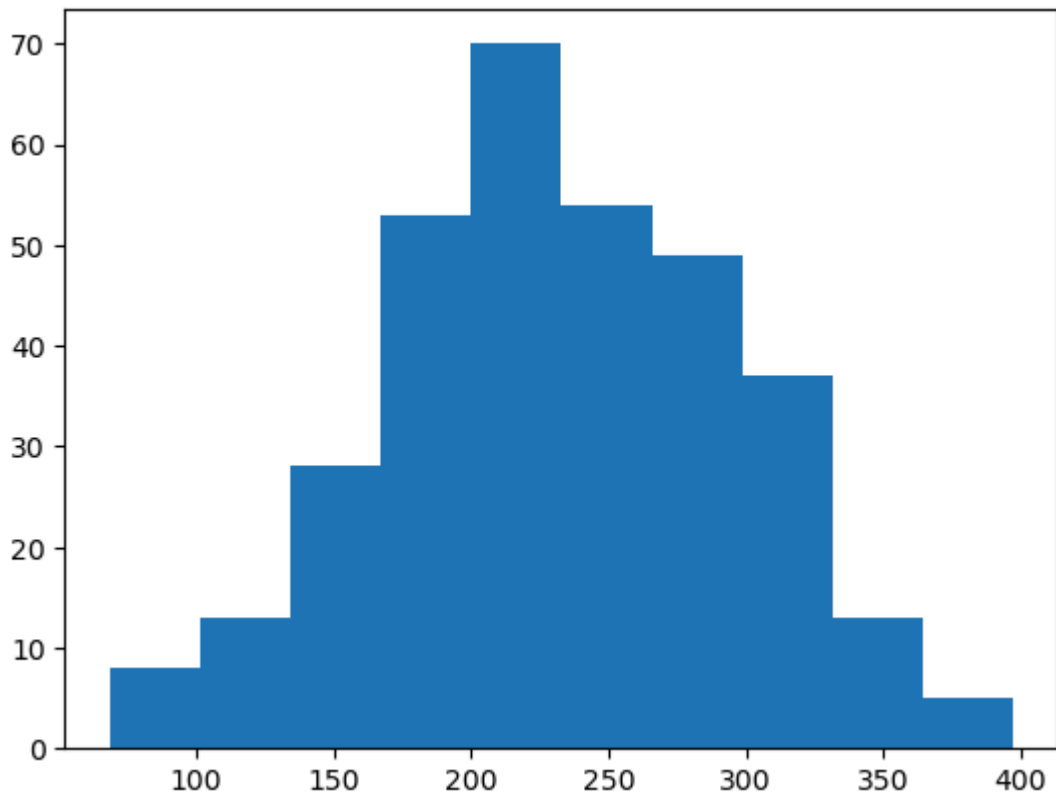


Histograms :

A histogram is a graph showing frequency distributions.

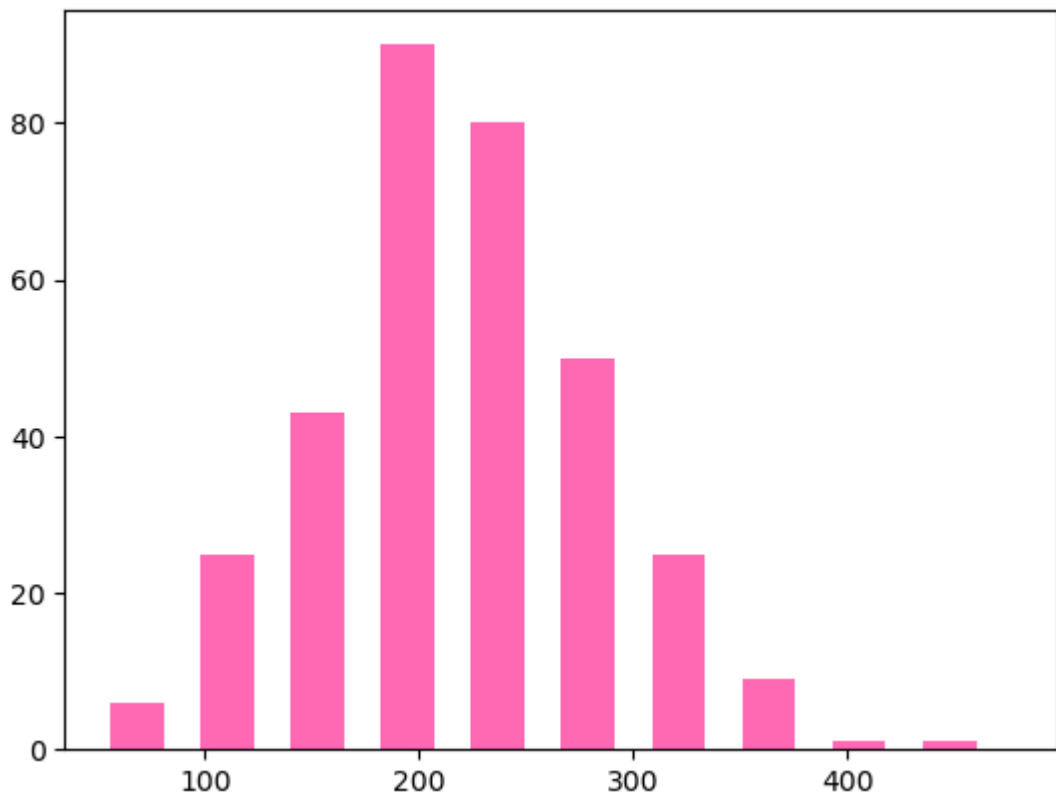
It is a graph showing the number of observations within each given interval.

```
In [96]: x = np.random.normal(230,60,330) # normal() takes at most 3 positional argument  
  
plt.hist(x)  
plt.show()
```



Histograms with color,width :

```
In [100... x = np.random.normal(230,60,330) # normal() takes at most 3 positional argument  
plt.hist(x, color = 'hotpink',width = 25) # Here width range is start from 1.  
plt.show()
```

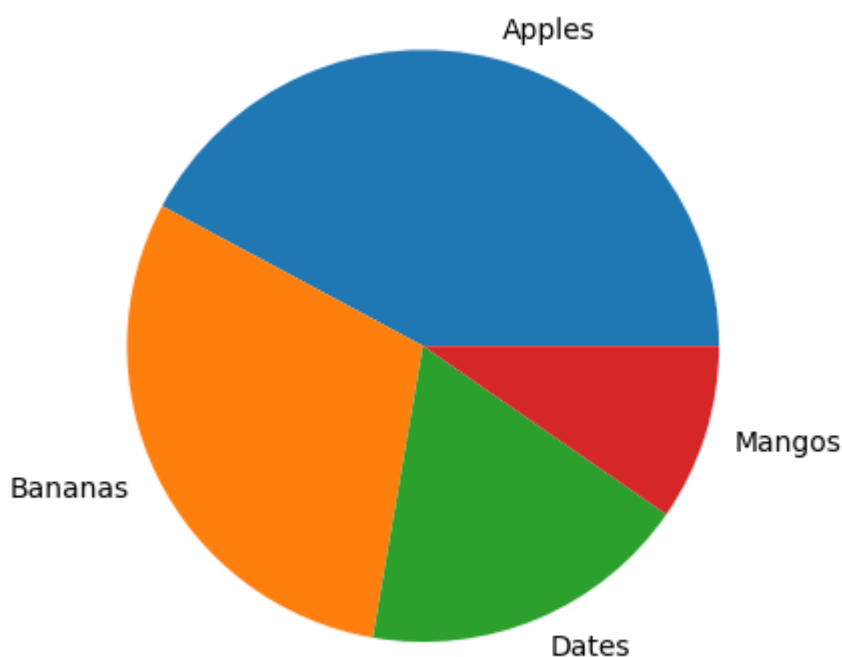


Pie Charts

1) Simple pie chart

```
In [102... A = np.array([350,250,150,80])
mAlabels = ['Apples', 'Bananas', 'Dates', 'Mangos']

plt.pie(A, labels = mAlabels)
plt.show()
```



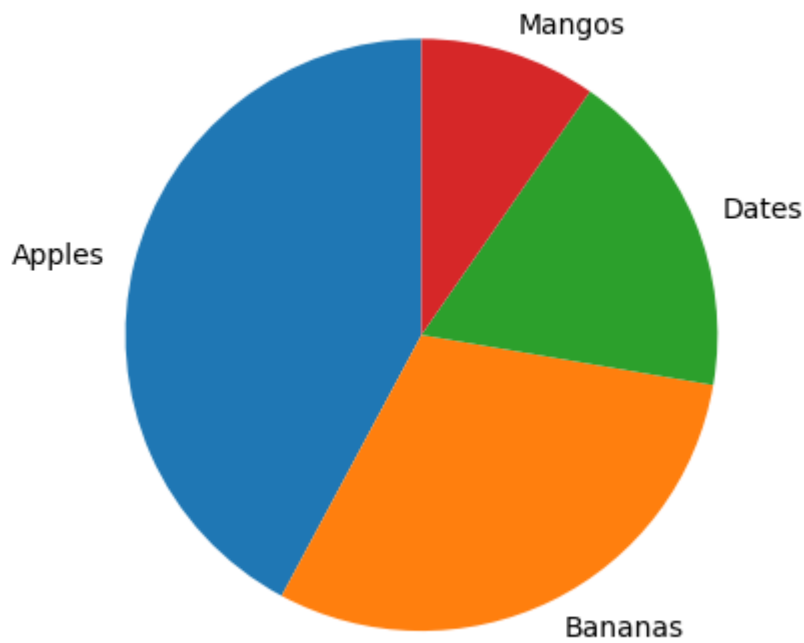
Start Angle :

The startangle parameter is defined with an angle in degrees, default angle is 0:

start angle at the x-axis and start anti-clockwise.

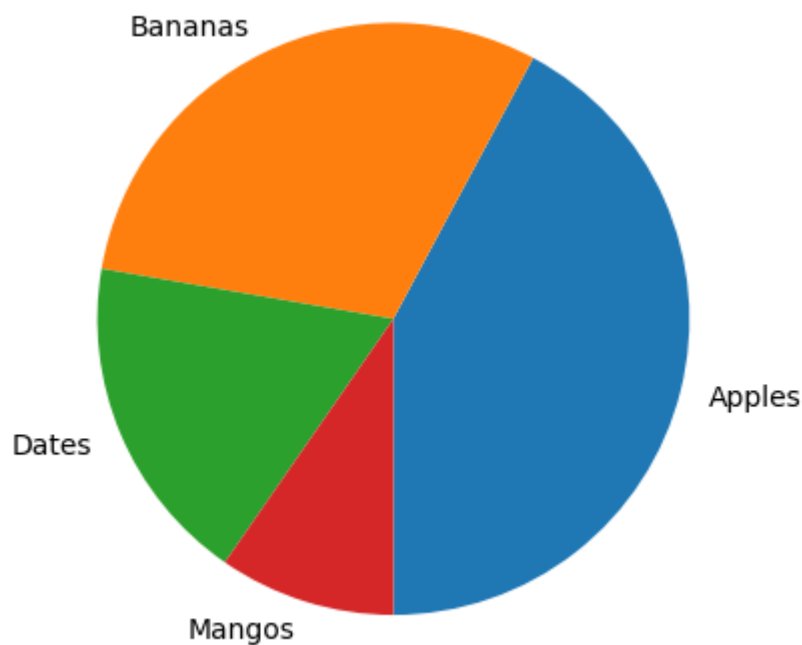
```
In [103... A = np.array([350,250,150,80])
mAlabels = ['Apples', 'Bananas', 'Dates', 'Mangos']

plt.pie(A, labels = mAlabels, startangle = 90)    # Start angle is started from
plt.show()
```



In [104...

```
A = np.array([350,250,150,80])  
mAlabels = ['Apples', 'Bananas', 'Dates', 'Mangos']  
  
plt.pie(A, labels = mAlabels, startangle = 270)    # Start angle is started from  
plt.show()
```

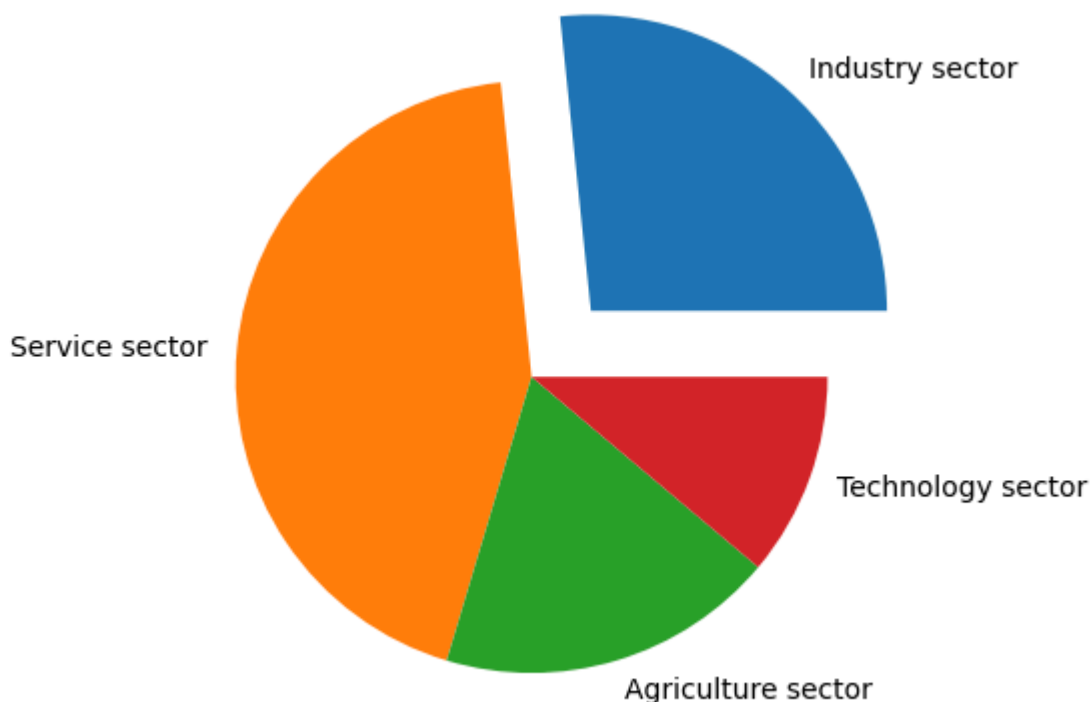


Explode pie chart:

In this explode pie one wedge to stand out

Each value represents how far from the center each wedge is displayed:

```
In [106... B = np.array([36,59,25,15])
mBlabels = ['Industry sector','Service sector','Agriculture sector','Technology
mBexplode = [0.3,0,0,0]
plt.pie(B, labels = mBlabels, explode = mBexplode)
plt.show()
```

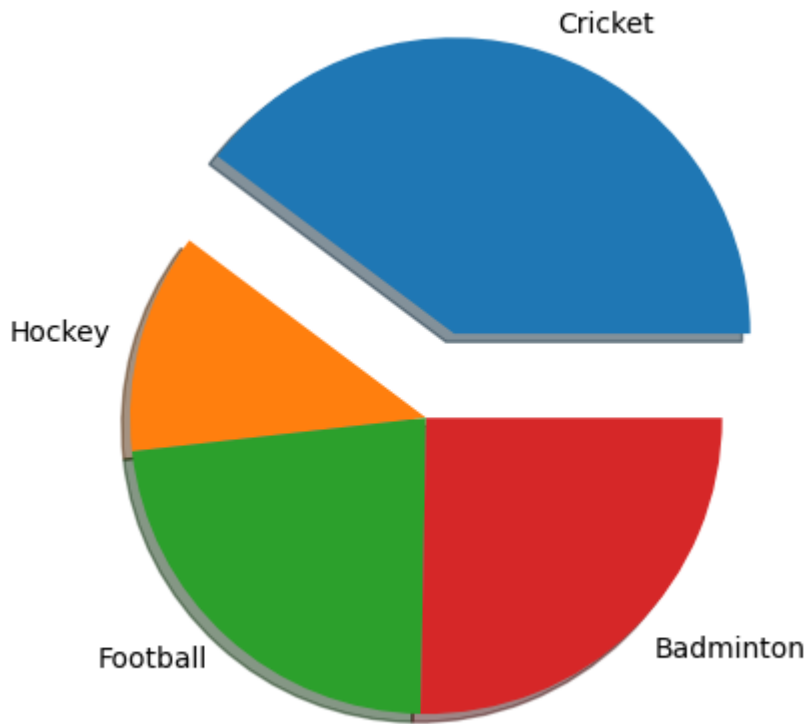


Shadow pie chart:

Add a shadow to the pie chart by setting the shadows parameter to True.

shadow = True

```
In [111... B = np.array([66,20,38,42])
mBlabels = ['Cricket','Hockey','Football','Badminton']
mBexplode = [0.3,0,0,0]
plt.pie(B, labels = mBlabels, explode = mBexplode, shadow = True)
plt.show()
```

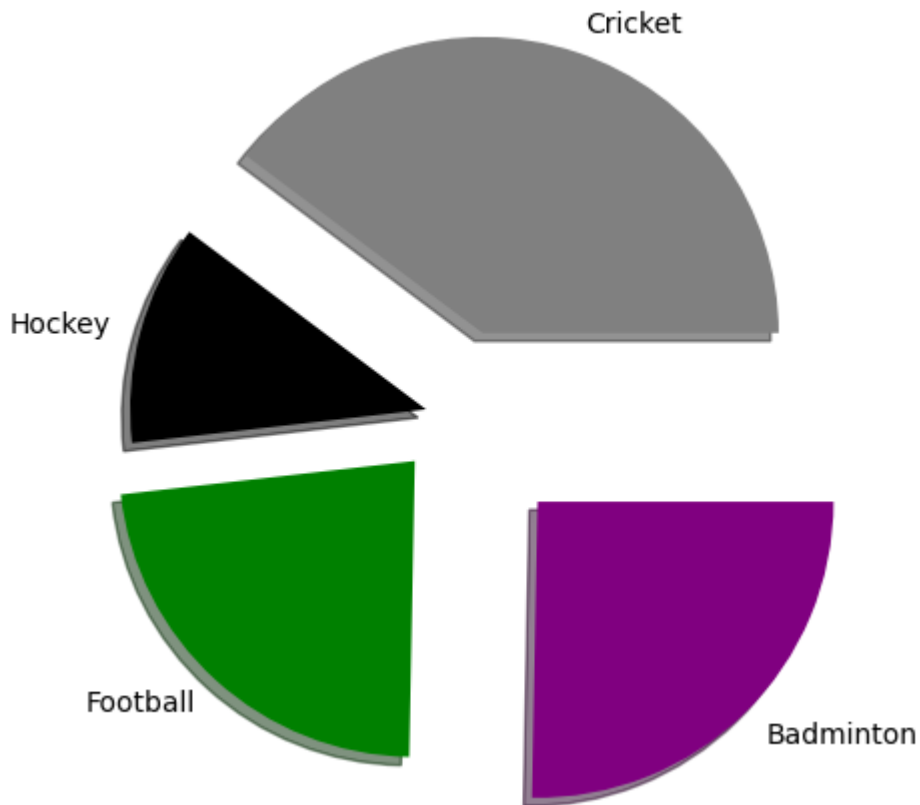


Color pie chart:

with explode, shadow.

Specify a new color for each wedge.

```
In [122... B= np.array([66,20,38,42])
mBlabels = ['Cricket','Hockey','Football','Badminton']
mBcolors = ['gray','black','g','purple'] # here we use also color code of 1st
mBexplode = [0.3,0.1,0.2,0.4]
plt.pie(B, labels = mBlabels, explode = mBexplode, shadow = True, colors = mBcol
plt.show()
```

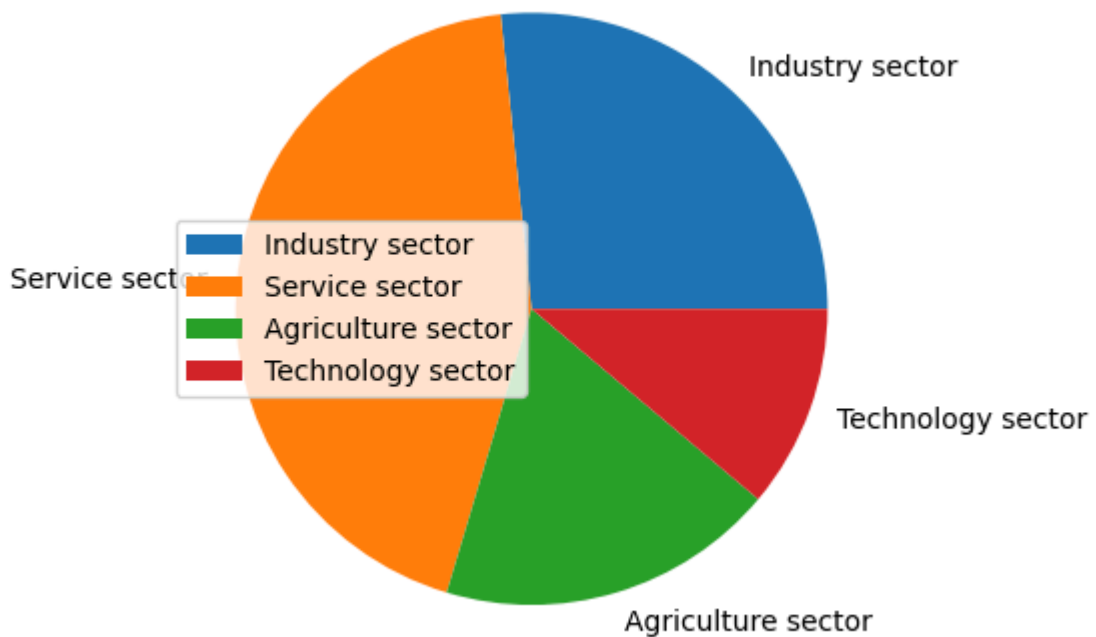


Add A Legend:

To add a list of explanation for each wedge, use the `legend()` function:

In [128...

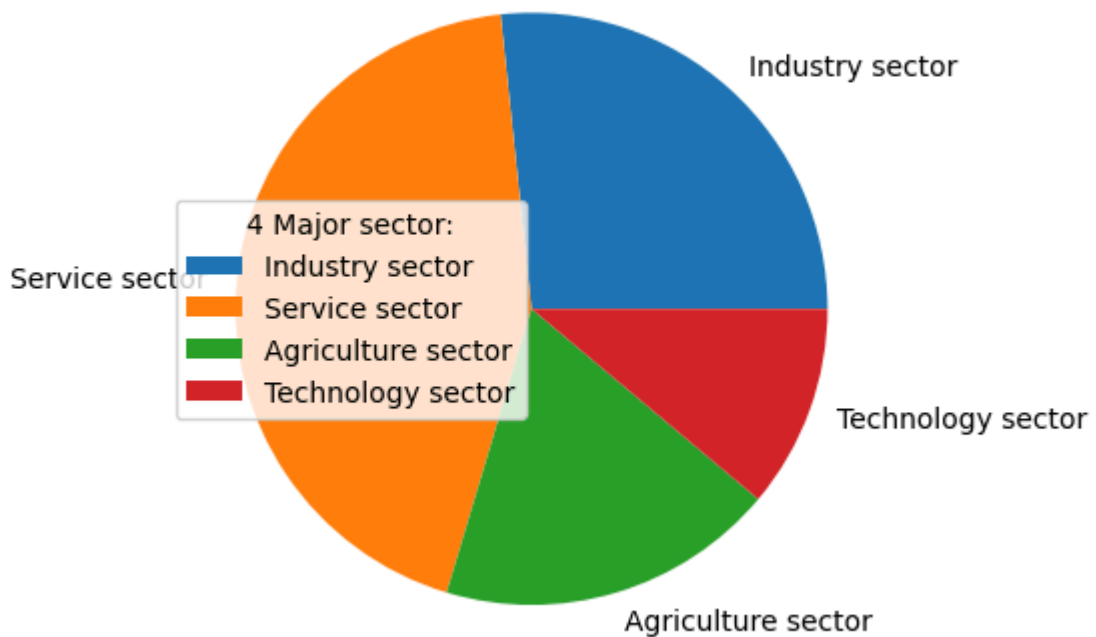
```
B = np.array([36,59,25,15])
mBlabels = ['Industry sector','Service sector','Agriculture sector','Technology']
mBexplode = [0.3,0,0,0]
plt.pie(B, labels = mBlabels)
plt.legend()
plt.show()
```

Add a legend with a header:

In [129...

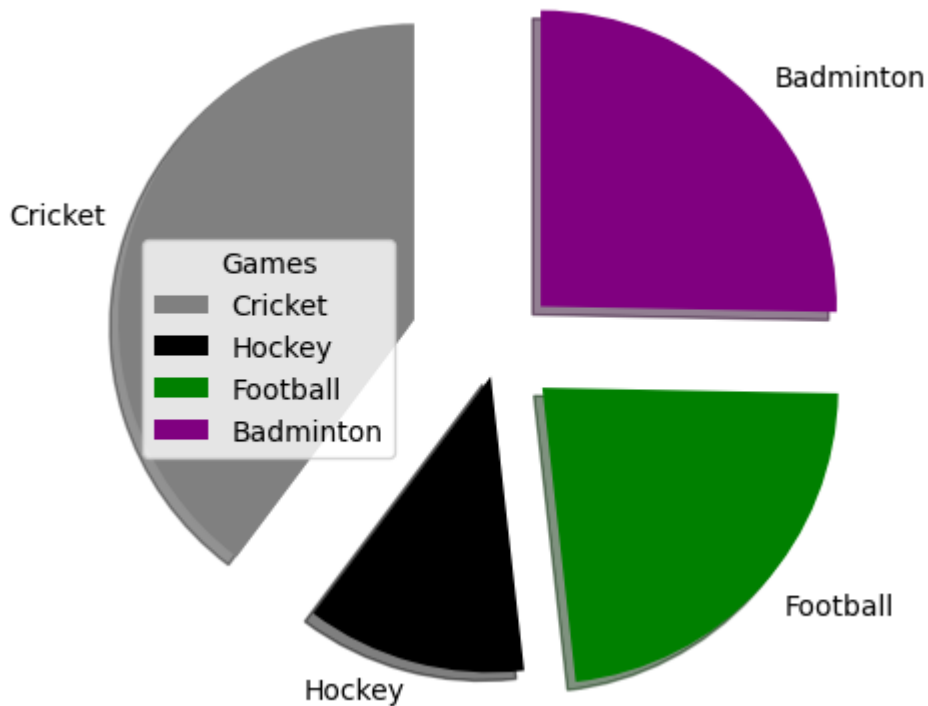
```
B = np.array([36,59,25,15])
mBlabls = ['Industry sector','Service sector','Agriculture sector','Technology
mBexplode = [0.3,0,0,0]
plt.pie(B, labels = mBlabls)
plt.legend(title = '4 Major sector:')
plt.show()
```



All function in one pie chart:

start angle,explode,shadow,color,labels,title.

```
In [132... B= np.array([66,20,38,42])
mBlabels = ['Cricket','Hockey','Football','Badminton']
mBcolors = ['gray','black','g','purple'] # here we use also color code of 1st
mBexplode = [0.3,0.1,0.2,0.2]
plt.pie(B, labels = mBlabels, explode = mBexplode, shadow = True,startangle = 90
plt.legend(title = 'Games')
plt.show()
```



In []: